

查找

难度	考点
3	多维数组（结构体）快排，二分查找的变式

本题思路较为简单，将 n 个数 x 保存到二维数组中的第一维，将输入顺序储存到第二维。对二维数组进行快排，第一位不同时按第一维排序，第一维相同时按第二维排序。排好序后进行二分查找，查找其左边界。如能查找到结果，则左边界元素的第二维就是该元素第一次出现的位置。二分查找的实现上有些难度。

易错点提示

以下是二分查找查找左边界的函数代码：

```
int left_bound(int nums[][2], int target, int len)
{
    if (len == 0)
        return -1;
    int left = 0;
    int right = len;
    while (left < right)
    {
        int mid = (left + right) / 2;
        if (nums[mid][0] == target)
        {
            right = mid;
        }
        else if (nums[mid][0] < target)
        {
            left = mid + 1;
        }
        else if (nums[mid][0] > target)
        {
            right = mid;
        }
    }
    if (left == len)
        return -1;
    return (nums[left][0] == target) ? left : -1;
}
```

需要注意其中的细节，如返回-1 的条件，left 和 right 状态的改变，循环的条件等。

代码

```

#include <stdio.h>
#include <stdlib.h>
int n, i, j, n1;
int x[1000005][2], temp;
int left_bound(int nums[][2], int target, int len)
{
    if (len == 0)
        return -1;
    int left = 0;
    int right = len;
    while (left < right)
    {
        int mid = (left + right) / 2;
        if (nums[mid][0] == target)
        {
            right = mid;
        }
        else if (nums[mid][0] < target)
        {
            left = mid + 1;
        }
        else if (nums[mid][0] > target)
        {
            right = mid;
        }
    }
    if (left == len)
        return -1;
    return (nums[left][0] == target) ? left : -1;
}

int cmp(const void *p1, const void *p2)
{
    int *m = (int *)p1;
    int *n = (int *)p2;
    if (*m != *n)
    {
        return *m - *n;
    }
    else
    {
        return *(m + 1) - *(n + 1);
    }
}

int read()
{
    char c;
    int flag = 0, num = 0;
    while ((c = getchar()) < '0' || c > '9')
        if (c == '-')
            flag = 1;
    while (c >= '0' && c <= '9')

```

```

    {
        num = num * 10 + c - 48;
        c = getchar();
    }
    return flag ? -num : num;
}
int main()
{
    n = read();
    for (i = 0; i < n; i++)
    {
        x[i][0] = read();
        x[i][1] = i + 1;
    }
    qsort(x, n, 8, cmp);
    while (~scanf("%d", &n1))
    {
        temp = left_bound(x, n1, n);
        if (temp == -1)
        {
            printf("NO\n");
        }
        else
        {
            printf("%d\n", x[temp][1]);
        }
    }
    return 0;
}

```

时间复杂度

$(m + n)\log n$

AUTHOR 包新蕾